

## Exercise Set 1: Class Statement, Private, Public, Constructor and Their Types

---

### Easy Exercises (1-5)

#### Exercise 1 Easy

Create a class `Rectangle` with private data members `length` and `width`. Include public member functions to set and get these values.

##### Expected Implementation Details:

The class should have two private double members for length and width. Implement public methods: `setLength(double l)`, `setWidth(double w)`, `getLength()`, and `getWidth()`. Add validation in setter methods to ensure positive values only. Include a method `calculateArea()` that returns `length × width`. Test by creating objects and verifying that direct access to length and width is not possible from outside the class.

## Exercise 2

Easy

Write a class `Student` with private members `name` (string) and `age` (int). Create a default constructor that initializes name to "Unknown" and age to 0.

### Expected Implementation Details:

Define private members as specified. The default constructor should be explicitly defined as `Student()` and initialize members in the constructor body or using initialization list. Add a public display method to show student details. Test by creating objects without arguments and verify the default values are set correctly. Consider what happens if you don't provide any constructor at all.

## Exercise 3

Easy

Modify the `Student` class to include a parameterized constructor that accepts name and age as parameters.

### Expected Implementation Details:

Keep the default constructor and add `Student(string n, int a)`. Use the initialization list for efficiency. Now the class has two constructors (overloaded). Add appropriate getter methods. Test by creating objects using both constructors. Verify that once you define any constructor, the compiler no longer provides a default constructor automatically.

#### Exercise 4 Easy

Create a class `BankAccount` with private member `balance` and public member functions `deposit()` and `getBalance()`. Ensure `balance` cannot be directly accessed from outside the class.

##### Expected Implementation Details:

Make `balance` a private double member. Implement `deposit(double amount)` that adds to `balance` only if amount is positive. Add `withdraw(double amount)` that checks sufficient balance before deducting. `getBalance()` should return current balance but not allow modification. Initialize `balance` to 0 in constructor. Test that `account.balance` gives compilation error while `account.getBalance()` works.

#### Exercise 5 Easy

Create a class `Circle` with a private member `radius`. Implement three constructors:

- Default constructor (`radius = 0`)
- Parameterized constructor accepting radius
- Copy constructor

##### Expected Implementation Details:

Implement `Circle()` setting `radius` to 0, `Circle(double r)` setting `radius` to `r`, and `Circle(const Circle& other)` copying `radius` from `other`. Add methods `getArea()` and `getCircumference()`. The copy constructor should use syntax: `radius(other.radius)` in initialization list. Test with: `Circle c1(5); Circle c2(c1);` Both should have same `radius` value.

## Intermediate Exercises (6-8)

### Exercise 6

#### Intermediate

Create a class `Employee` with const member `employeeID` and reference member `department`. Use member initialization list in the constructor to initialize these members along with regular members `name` and `salary`.

#### Expected Implementation Details:

Declare `const int employeeID` and `string& department` as private members. These MUST be initialized using initialization list, not in constructor body. Constructor signature: `Employee(int id, string& dept, string n, double sal)`. The initialization list syntax: `: employeeID(id), department(dept), name(n), salary(sal) {}`. Test by creating a string variable for department and passing it by reference. Verify that `employeeID` cannot be modified after construction.

### Exercise 7

Intermediate

Design a class `Date` with private members `day`, `month`, `year`. Implement:

- Default constructor (sets to current date)
- Parameterized constructor with validation (e.g., month 1-12, appropriate days per month)
- Copy constructor

Include a display function that shows date in DD/MM/YYYY format.

#### Expected Implementation Details:

Implement validation: months 1-12, days 1-31 (considering month), handle leap years for February. Default constructor can set to 01/01/2024 or use system date. Parameterized constructor should set to safe defaults if invalid input provided. Include private helper `isLeapYear(int year)` and `daysInMonth(int month, int year)`. Display format: 05/03/2024 with leading zeros. Test with invalid dates like 31/02/2024.

### Exercise 8

Intermediate

Create a class `DatabaseConnection` that ensures only one instance can be created (singleton pattern basics). Use a private constructor and a public static method `getInstance()` to control object creation.

#### Expected Implementation Details:

Make constructor private: `DatabaseConnection()`. Add private static pointer: `static DatabaseConnection* instance`. Implement `static DatabaseConnection* getInstance()` that creates instance on first call and returns existing instance on subsequent calls. Include a `connect()` method to simulate connection. Prevent copying by making copy constructor private. Test: multiple calls to `getInstance()` should return same object address.

## Difficult Exercises (9-10)

### Exercise 9 Difficult

Design a class hierarchy: `Person` (base) and `Employee` (derived).

- `Person` has private members: name, age, and ID (const)
- `Employee` has private members: salary, department, and employee code
- Implement all types of constructors in both classes
- Use initialization lists properly
- Demonstrate constructor chaining from derived to base class
- Include a static member to auto-generate IDs

#### Expected Implementation Details:

In `Person`: `static int nextID` to generate unique IDs, const member `ID` must be initialized in initialization list. `Person` constructors: default (age=0, name=""), parameterized, and copy. `Employee` must call `Person` constructor explicitly:

```
Employee(...) : Person(name, age), salary(s), department(d),  
employeeCode(ec) {}
```

Each `Person` object gets unique auto-incremented ID. `Employee` inherits `Person`'s public methods. Test complete initialization chain and verify ID generation across multiple objects.

## Exercise 10

Difficult

Create a class `SmartArray` that:

- Manages a dynamic integer array (private pointer member)
- Implements parameterized constructor (accepts size)
- Implements copy constructor with deep copy
- Implements destructor to prevent memory leaks
- Has private member functions for memory allocation/deallocation
- Includes public methods: `setValue()`, `getValue()`, `getSize()`
- Bonus: Implement move constructor (C++11)

### Expected Implementation Details:

Private members: `int* data` and `int size`. Constructor allocates: `data = new int[size]`. Copy constructor must allocate new array and copy elements, not just pointer. Destructor: `delete[] data`. Include bounds checking in `setValue/getValue`. Private helper `allocateMemory(int s)` and `deallocateMemory()`. RAII principle: resource acquired in constructor, released in destructor. Test deep copy: modify copy without affecting original. Move constructor (bonus) transfers ownership: `data = other.data; other.data = nullptr`.